

SYSTEM GENRL GAMEMASTER AI – DOKUMENTACJA SDLC

Niniejsza dokumentacja techniczna opisuje założenia projektowe, architektoniczne oraz plan wdrożenia zintegrowanego systemu **GenRL GameMaster AI**. Jest to system klasy neuroewolucyjnej, przeznaczony do automatycznej nauki zasad i optymalizacji strategii rozgrywki w środowiskach gier dwuwymiarowych (2D Simulator, np. *Super Mario*, *Pacman*). System łączy techniki uczenia ze wzmocnieniem (Reinforcement Learning) z meta-optymalizacją realizowaną przez algorytmy genetyczne (GA), wykorzystując wieloagentową orkiestrację LLM/LRM.

1. PRD (Product Requirements Document) – Dokument Wymagań Produktowych

1.1. Cel i wizja produktu

Głównym celem systemu **GenRL GameMaster AI** jest stworzenie samouczącego się agenta, który bez uprzedniej znajomości reguł gry (tablic przejść, logiki punktacji czy map kolizji) potrafi opanować środowisko gry 2D wyłącznie na podstawie stanów końcowych (wygrana/przegrana) oraz sygnałów rzadkich nagród. Kluczową innowacją produktu jest zastosowanie pętli algorytmu genetycznego nadzorowanej przez modele LRM (Large Reasoning Models). Modele te analizują dotychczasowy przebieg ewolucji kodu oraz wag sieci, a następnie modyfikują strukturę kodu algorytmu i hiperparametry uczenia w celu przełamania lokalnych minimów wydajności.

1.2. Grupa docelowa (Persony)

- Badacz AI / Inżynier R&D (Persona: Badacz):** Osoba poszukująca elastycznego środowiska do testowania hybrydowych algorytmów ewolucyjno-wzmocnieniowych. Wymaga pełnego wglądu w logi mutacji, strukturę populacji genomów oraz wykresy zbieżności funkcji przystosowania.
- Inżynier QA / Tester Automatyczny (Persona: Weryfikator):** Specjalista wykorzystujący autonomicznego agenta do wykrywania błędów (glitchy, niewidzialnych ścian, luk w mapach) w nowo projektowanych grach 2D poprzez masową, nieludzką eksplorację poziomów.

1.3. Kluczowe funkcjonalności (Features) i wymagania produktowe

- Automatyczny Parser Środowiska i Stanu:** Moduł odczytujący macierze pikseli lub wektory cech z symulatora 2D gier *Super Mario* / *Pacman* i mapujący je na przestrzeń stanów RL.
- Orkiestrator Ewolucyjny (LangGraph Loop):** Graf stanów sterujący cyklem życia populacji agentów. Odpowiada za ewaluację pokolenia, selekcję, mutację kodu algorytmicznego oraz krzyżowanie parametrów.
- Odizolowany Moduł Wykonawczy (OpenCode Sandbox):** Bezpieczny komponent uruchamiający zmodyfikowany kod Pythona/C++ odpowiedzialny za interakcję agenta z grą.
- Kaskada Przetwarzania Kognitywnego (Hermes 3 / Codex):** System inteligentnego generowania zmian w kodzie nagrody (Reward Shaping) i mutacji struktur sieci przy użyciu modelu Hermes 3, z redundantnym przełączeniem awaryjnym na OpenAI Codex.
- Instytucjonalny Pulpit Kontrolny (NBP-Style Dashboard):** Wizualna warstwa prezentacji danych oparta na rygorystycznym wzornictwie finansowym, eliminująca elementy rozpraszające na rzecz surowej analizy metryk zbieżności.

1.4. Konkretnie User Stories (Opowieści Użytkowników)

US-01 Inicjalizacja Autonomicznego Treningu w Środowisku 2D

Jako Badacz AI, *chcę* wybrać rodzaj symulatora (np. Pacman) z poziomu interfejsu i zdefiniować bazową funkcję celu, *aby* system uruchomił pierwszą generację agentów uczących się zasad gry od zera.

Kryteria akceptacji: System poprawnie ładuje instancję gry w tle, inicjalizuje bazową macierz Q-learningu lub wagi sieci neuronowej i loguje pierwszy krok epoki w bazie PostgreSQL w czasie krótszym niż **t < 3.0s**.

US-02 Modyfikacja Kodu Algorytmu przez Agenta LRM

Jako Inżynier R&D, *chcę*, aby w momencie stagnacji wyników (brak wzrostu średniej nagrody przez 50 generacji) model Hermes 3 przeanalizował historię i wygenerował nową mutację algorytmu genetycznego,

Kryteria akceptacji: System przekazuje dotychczasowy przebieg funkcji przystosowania jako kontekst do OpenRouter (model Hermes 3), otrzymuje spójną semantycznie poprawkę kodu algorytmu i wyświetla pełny proces myślowy (reasoning tokens) w dedykowanym panelu w stylistyce złota NBP.

US-03 Bezpieczna Egzekucja Kodu Poprzez OpenCode Worker

Jako Administrator Systemu, *chcę*, aby wygenerowany przez LLM dynamiczny kod mutacji algorytmu był uruchamiany przez robotnika OpenCode w izolowanym kontenerze,

Kryteria akceptacji: Każda próba ucieczki z piaskownicy (np. wywołania systemowe, operacje na plikach hosta) skutkuje przerwaniem wątku, zwróceniem kodu błędu do PostgreSQL i powiadomieniem modułu orkiestracji o konieczności korekty kodu.

US-04 Odporność na Błędy API poprzez Kaskadę OpenAI Codex

Jako Inżynier DevOps, *chcę*, aby w przypadku niedostępności platformy OpenRouter, zapytania o generowanie nowego kodu były kierowane do API OpenAI Codex z użyciem zapasowego klucza ChatGPT API,

Kryteria akceptacji: Przełączenie następuje w sposób przezroczysty dla trwającego eksperymentu ewolucyjnego, a informacja o failoverze jest trwale zapisywana w logu audytowym bazy PostgreSQL.

1.5. Wymagania нефunkcjonalne

- **Izolacja środowiska:** Czas życia kontenera OpenCode Worker dla pojedynczej ewaluacji osobnika ograniczony do maksymalnie 60 sekund w celu przeciwdziałania pętlom nieskończonym.
- **Wydajność bazy danych:** Zapis stanów gry i parametrów chromosomów optymalizowany pod kątem przepustowości do 500 wpisów na sekundę przy użyciu mechanizmu unifikacji transakcji (bulk inserts) w PostgreSQL.
- **Zgodność wizualna:** Interfejs frontendu musi ściśle implementować paletę kolorystyczną serwisu nbp.pl, zachowując kontrast WCAG na poziomie minimum 4.5:1.

1.6. Kryteria sukcesu i kluczowe metryki (KPI)

IDENTYFIKATOR	NAZWA METRYKI (KPI)	WARTOŚĆ DOCELOWA	METODOLOGIA POMIARU
KPI-01	Wskaźnik Konwergencji Ewolucyjnej	> 90% przypadków	Osiągnięcie stabilnego poziomu przejścia 1. etapu gry (Super Mario/Pacman) przed 200. generacją algorytmu genetycznego.

IDENTYFIKATOR	NAZWA METRYKI (KPI)	WARTOŚĆ DOCELOWA	METODOLOGIA POMIARU
KPI-02	Efektywność Pracy Robotnika (OpenCode Worker)	< 5% odrzuceń syntaktycznych	Stosunek kodu poprawnie skompilowanego/ uruchomionego w kontenerze do całkowitej liczby mutacji generowanych przez model LRM.
KPI-03	Stabilność Kaskady LLM (Failover Latency)	< 4.0 sekundy	Czas migracji kontekstu i uzyskania poprawki z OpenAI Codex po awarii węzła OpenRouter.

2. ARD (Architectural Reference Document) – Architektoniczny Dokument Referencyjny

2.1. Wielojęzyczna (Poliglotyczna) Architektura Systemu

System został zaprojektowany w paradygmacie mikrousług rozproszonych, optymalizując dobór technologii do realizowanych zadań matematycznych i infrastrukturalnych:

WARSTWA / KOMPONENT	TECHNOLOGIA	UZASADNIENIE WYBORU I ROLA ARCHITEKTONICZNA
Prezentacja i Monitoring	React.js / TypeScript	Odpowiada za rendering siatki symulatora 2D oraz wizualizację drzewa genealogicznego algorytmu genetycznego. Stylistyka i komponenty nawiązują bezpośrednio do serwisu nbp.pl .
Orkiestracja i Zarządzanie Stanem AI	Python / LangGraph / LangChain	Rdzeń logiczny systemu. LangGraph służy do implementacji stanowej pętli ewolucyjnej (State: <i>Population</i> -> <i>Evaluate</i> -> <i>Mutate via LLM</i> -> <i>Next Generation</i>). LangChain unifikuje dostęp do OpenRouter i OpenAI.
Integracja Symulatora i Silnik Analizy Strukturalnej	Java / LangChain4j / Spring Boot	Java zapewnia stabilne, wielowątkowe środowisko do uruchamiania i kontrolowania instancji emulatorów gier (często napisanych w Javie lub wymagających precyzyjnego zarządzania pamięcią). LangChain4j służy do strukturalnego mapowania genomów i walidacji typów parametrów algorytmu przed egzekucją.
BFF & Stream Gateway	Node.js / Express / WebSockets	Odpowiada za niskopoziomowe, asynchroniczne przesyłanie klatek obrazu z symulatora (Java/Python) bezpośrednio do przeglądarki użytkownika (React) oraz zarządzanie połączeniami API.
Trwała Warstwa Danych	PostgreSQL (v16)	Relacyjna baza danych przechowująca pełne sekwencje chromosomów, parametry wagowe sieci neuronowych, tablice Q-wartości, logi sesji LRM oraz metryki wydajnościowe generacji.

2.2. Topologia Agentowa i Pętla Reasoning-Evolution

Logika podejmowania decyzji i transformacji kodu opiera się na cyklicznym grafie agentowym zarządzanym przez komponent Pythona. Proces ewolucyjny realizowany jest według następującej sekwencji logicznej:

- **Krok 1 (Ewaluacja w Java/OpenCode):** Środowisko Java uruchamia symulator 2D. Kod sterujący (wygenerowany w poprzednim kroku) wykonuje akcje w grze. Wyniki (punkty, czas przeżycia, status wygranej) są agregowane i zapisywane w bazie PostgreSQL.

- **Krok 2 (Analiza Stagnacji w LangGraph):** Agent monitorujący analizuje dane w PostgreSQL. Jeśli wykryty zostanie brak postępu, następuje aktywacja węzła wnioskowania (Reasoning Node).
- **Krok 3 (Generowanie Mutacji przez Hermes 3):** Poprzez platformę **OpenRouter**, model **Hermes 3** otrzymuje aktualny kod algorytmu genetycznego oraz historię nieudanych prób. Wykorzystując zaawansowane tokeny wnioskowania (Reasoning), model dokonuje modyfikacji kodu (np. zmienia operatory krzyżowania, wprowadza mutacje adaptacyjne lub redefiniuje nagrody).
- **Krok 4 (Weryfikacja i Egzekucja):** Kod trafia do **OpenCode Worker**. Jeśli testy składniowe zakończą się sukcesem, nowy algorytm staje się podstawą dla kolejnej generacji agentów (Krok 1).

2.3. Standardy Projektowe Identyfikacji Wizualnej (Kanon NBP.pl)

Interfejs graficzny systemu odzwierciedla powagę instytucjonalną. Zgodnie z wytycznymi wzornictwa portalu Narodowego Banku Polskiego, wprowadza się następujące obostrzenia stylistyczne:

- **Kolorystyka Bazowa:** Dominujący głęboki granat korporacyjny (**#002C5B**) stosowany w nagłówkach, menu głównym oraz jako tło dla kluczowych wskaźników. Kolor uzupełniający to stonowane, matowe złoto (**#B59A57**) używane do obramowań, wyróżnień stanów aktywnych i akcentów nawigacyjnych. Tło ekranów roboczych to jasny krem (**#FCFBFA**).
- **Typografia i Oznaczenia:** Nagłówki główne wykorzystują monumentalną czcionkę szeryfową (Georgia/Times New Roman), natomiast dane liczbowe i tabele prezentowane są przy użyciu czystej czcionki bezszeryfowej (Arial/Helvetica). Logotyp systemu musi zawierać uproszczony, geometryczny sygnet instytucjonalny z wyraźnym, klasycznym liternictwem.

3. PLAN (Plan Projektowy i Harmonogram SDLC)

Wdrożenie systemu zaplanowano na okres 6 miesięcy w oparciu o zwinny cykl wytwórczy (Scrum, sprinty 2-tygodniowe), zapewniający ciągłą walidację wyników ewolucyjnych.

3.1. Faza 1: Inicjacja, Modelowanie Matematyczne i UX NBP (Miesiąc 1-2)

- **Projektowanie Interfejsu:** Opracowanie kompletnej księgi znaków i makiet w programie Figma z uwzględnieniem surowej palety kolorystycznej i typograficznej NBP.pl.
- **Architektura Danych:** Implementacja schematu bazy danych w PostgreSQL (tabele dla chromosomów, logów agentów i stanów środowiska).
- **Integracja Bazowa Symulatorów:** Przygotowanie środowiska Java do uruchamiania gier **Super Mario** oraz **Pacman** w trybie bezgłowym (headless mode).

3.2. Faza 2: Faza Wdrożeniowa i Implementacja Komponentów (Miesiąc 3-4)

Równoległe prace programistyczne podzielone na dedykowane bloki technologiczne:

ETAP (SPRINTY)	DOMENA TECHNOLOGICZNA	ZAKRES PRAC I KAMIENIE MIŁOWE (MILESTONES)
Sprint 1 - 2	Infrastruktura Podkładowa i Node.js Gateway	Konfiguracja potoków automatyzacji CI/CD (GitHub Actions). Implementacja serwera BFF w Node.js, uruchomienie komunikacji przez WebSockets do przesyłania stanów gier. Implementacja layoutu wizualnego NBP w React.js.

ETAP (SPRINTY)	DOMENA TECHNOLOGICZNA	ZAKRES PRAC I KAMIENIE MIŁOWE (MILESTONES)
Sprint 3 - 4	Rdzeń Agentowy AI (Python / LangGraph)	Implementacja grafu stanów ewolucyjnych w LangGraph. Integracja z OpenRouter celem komunikacji z modelem Hermes 3. Opracowanie struktur promptów dla mutacji algorytmu genetycznego. Zapewnienie obsługi tokenów myślowych.
Sprint 5 - 6	Warstwa Java (LangChain4j) & OpenCode	Uruchomienie mikrousługi Java walidującej strukturalną poprawność kodu. Integracja z piaskownicą robotnika OpenCode. Implementacja mechanizmu przezroczystego przełączania awaryjnego (failover) do OpenAI Codex.

3.3. Faza 3: Testy Zaawansowane, Optimalizacja i Jakość (Miesiąc 5)

- **Testy Stabilności Algorytmicznej:** Weryfikacja odporności systemu na zjawisko przedwczesnej zbieżności algorytmu genetycznego (Genetic Drift).
- **Audyt Bezpieczeństwa OpenCode:** Przeprowadzenie zaawansowanych testów penetracyjnych izolacji kontenerów pod kątem prób eskalacji uprawnień przez wygenerowany kod autonomiczny.
- **Zamknięte Testy Beta:** Udostępnienie platformy wybranemu zespołowi 500 inżynierów i badaczy w celu oceny czytelności procesu wnioskowania oraz spójności interfejsu NBP-Style.

3.4. Faza 4: Wdrożenie Produkcyjne i Utrzymanie (Miesiąc 6)

- **Wdrożenie Środowiska (Release):** Osadzenie systemu na klastrze Kubernetes z pełną konteneryzacją usług Node.js, Python, Java oraz bazy PostgreSQL.
- **Telemetria i Nadzór:** Konfiguracja systemów Datadog oraz Firebase Performance pod kątem monitorowania opóźnień generowania tokenów ewolucyjnych oraz utraty połączeń z API zewnętrznymi.
- **Procedury Utrzymeniowe:** Przekazanie systemu do eksploatacji, konfiguracja automatycznych kopii zapasowych baz PostgreSQL zawierających wyewoluowane, optymalne genomy agentów.